

slice

July 14, 2025

```
[36]: import matplotlib.pyplot as plt
import numpy as np
from scipy import interpolate
```

```
[37]: # Load coordinate and curvature data
x, y = np.loadtxt('RuO2-kslice-coord.dat', unpack=True)
z = np.loadtxt('RuO2-kslice-curv.dat', unpack=True)

# Define k-slice plane according to your specification:
# kslice_corner = 0.0 0.0 0.0      ! Gamma point (corner 1)
# kslice_b1 = 0.5 0.5 0.0          ! Direction to M point
# kslice_b2 = 0.0 0.0 0.5          ! Direction to A point (via Z)

# K-point coordinates in fractional coordinates
gamma_point = np.array([0.0, 0.0, 0.0]) #  $\Gamma$  point
M_point = np.array([0.5, 0.5, 0.0])     # M point
A_point = np.array([0.0, 0.0, 0.5])     # A point

print(f>Data shape: x={len(x)}, y={len(y)}")
print(f>Berry curvature components: {len(z)} (x, y, z components)")
print(f>X range: {x.min():.3f} to {x.max():.3f}")
print(f>Y range: {y.min():.3f} to {y.max():.3f}")
```

Data shape: x=9409, y=9409

Berry curvature components: 3 (x, y, z components)

X range: 0.000 to 0.979

Y range: 0.000 to 1.003

```
[38]: # Create figure with subplots for all three Berry curvature components
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Component names and data
components = ['x', 'y', 'z']
component_data = [z[0], z[1], z[2]]

for i, (ax, comp_name, comp_data) in enumerate(zip(axes, components,
↪component_data)):
    # Create scatter plot with improved colormap
```

```

scatter = ax.scatter(x, y, c=comp_data, cmap='RdBu_r', s=20, alpha=0.8)

# Add colorbar
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label(f'Berry Curvature ({comp_name})', fontsize=12)

# Set k-point labels and ticks
ax.set_xlabel('k-direction ( $\Gamma \rightarrow M$ )', fontsize=12)
ax.set_ylabel('k-direction ( $\Gamma \rightarrow Z$ )', fontsize=12)
ax.set_title(f'Berry Curvature ({comp_name}-component)', fontsize=14)

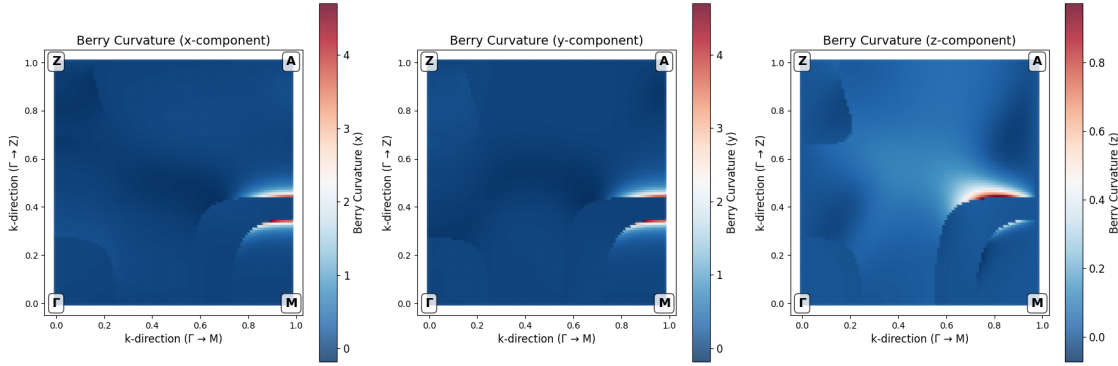
# Add k-point labels at corners if data spans the full range
x_range = x.max() - x.min()
y_range = y.max() - y.min()

# Add k-point markers and labels
if x_range > 0.3 and y_range > 0.3: # Only if we have significant range
    ax.text(x.min(), y.min(), ' $\Gamma$ ', fontsize=14, fontweight='bold',
            ha='center', va='center', bbox=dict(boxstyle="round,pad=0.3",
            ↪facecolor="white", alpha=0.8))
    ax.text(x.max(), y.min(), 'M', fontsize=14, fontweight='bold',
            ha='center', va='center', bbox=dict(boxstyle="round,pad=0.3",
            ↪facecolor="white", alpha=0.8))
    ax.text(x.min(), y.max(), 'Z', fontsize=14, fontweight='bold',
            ha='center', va='center', bbox=dict(boxstyle="round,pad=0.3",
            ↪facecolor="white", alpha=0.8))
    ax.text(x.max(), y.max(), 'A', fontsize=14, fontweight='bold',
            ha='center', va='center', bbox=dict(boxstyle="round,pad=0.3",
            ↪facecolor="white", alpha=0.8))
    # Customize ticks to show fractional coordinates
    ax.tick_params(labelsize=10)

# Set equal aspect ratio
ax.set_aspect('equal')

plt.tight_layout()
plt.show()

```



```
[39]: # Create a detailed plot for a single component (z-component) with interpolated
      ↪ contours
plt.figure(figsize=(12, 10))

# Create a grid for interpolation
x_min, x_max = x.min(), x.max()
y_min, y_max = y.min(), y.max()

# Create regular grid
num_points = 100
x_grid = np.linspace(x_min, x_max, num_points)
y_grid = np.linspace(y_min, y_max, num_points)
X_grid, Y_grid = np.meshgrid(x_grid, y_grid)

# Plot interpolated contour maps for all three Berry curvature components
fig, axes = plt.subplots(1, 3, figsize=(22, 7))
# Use the same color scale for all components
vmin = min(np.min(z[0]), np.min(z[1]), np.min(z[2]))
vmax = max(np.max(z[0]), np.max(z[1]), np.max(z[2]))
levels = np.linspace(vmin, vmax, 50)
for idx, (ax, comp_name) in enumerate(zip(axes, components)):
    # Interpolate component data onto grid
    comp_interp = interpolate.griddata((x, y), z[idx], (X_grid, Y_grid),
    ↪method='cubic', fill_value=0)
    contour = ax.contourf(X_grid, Y_grid, comp_interp, levels=levels,
    ↪cmap='RdBu_r', extend='both')
    cbar = plt.colorbar(contour, ax=ax)
    cbar.set_label(f'Berry Curvature ({comp_name})', fontsize=12)
    ax.scatter(x, y, c=z[idx], cmap='RdBu_r', s=8, edgecolors='black',
    ↪linewidth=0.2, alpha=0.7)
    ax.set_xlabel('k-direction (Γ → M)', fontsize=12)
    ax.set_ylabel('k-direction (Γ → Z)', fontsize=12)
    ax.set_title(f'Berry Curvature ({comp_name}-component)', fontsize=14)
```

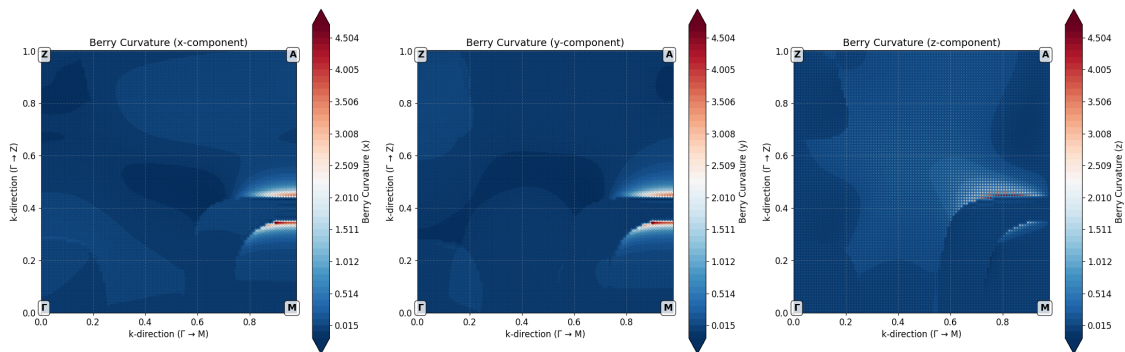
```

# Add k-point labels at corners
if x_range > 0.3 and y_range > 0.3:
    ax.text(x_min, y_min, ' $\Gamma$ ', fontsize=13, fontweight='bold', ha='left',
    ↪va='bottom',
    ↪bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.
    ↪8))
    ax.text(x_max, y_min, 'M', fontsize=13, fontweight='bold', ha='right',
    ↪va='bottom',
    ↪bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.
    ↪8))
    ax.text(x_min, y_max, 'Z', fontsize=13, fontweight='bold', ha='left',
    ↪va='top',
    ↪bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.
    ↪8))
    ax.text(x_max, y_max, 'A', fontsize=13, fontweight='bold', ha='right',
    ↪va='top',
    ↪bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.
    ↪8))
    ax.grid(True, alpha=0.3, linestyle='--')
    ax.set_aspect('equal')

plt.tight_layout()
plt.show()

```

<Figure size 1200x1000 with 0 Axes>



```
[40]: np.max(z[0]), np.max(z[1]), np.max(z[2])
```

```
[40]: (np.float64(4.7035835), np.float64(4.7022762), np.float64(0.97305419))
```

```
[ ]:
```